



OASIS Security Report

Debug Information Exposure Security Analysis

Date: 2026-06-11 10:07:08 — **Model:** mistral:latest

Vulnerability: Debug Information Exposure

Project: clean_contracts

Analysis root: `../clean`

Table of contents

▢ Summary

▢ Statistics

▢ Files

▢ Detailed analysis

▢ Assistant

△ Errors & notes

Summary

Analyzed **3** file(s) for Debug Information Exposure.

[Return to table of contents](#)

Statistics

Files analyzed: 3 **Total findings:** 3

Critical

(

0

High		0
Medium		3
Low		0

[Return to table of contents](#)

Files

File	Similarity
<code>/work/project/dataset/clean/simple_dao_fixed.sol</code>	0.581
<code>/work/project/dataset/clean/modifier_reentrancy_fixed.sol</code>	0.564
<code>/work/project/dataset/clean/mycontract_fixed.sol</code>	0.548

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/clean/simple_dao_fixed.sol

Similarity score: 0.581

Source lines: 1-34

► Finding 1.1: Vulnerability found (Medium)

Vulnerable code

```
withdraw(uint amount) public {  
    if (credit[msg.sender]>= amount) {  
        credit[msg.sender]-=amount;  
        require(msg.sender.call.value(amount));  
    }  
}
```

Explanation

Debugging information is exposed through the use of 'require' statement, which includes a stack trace. In this specific case, the call to the `msg.sender.call.value()` function will disclose that it is calling a contract function.

Impact

Information disclosure

Entry point

withdraw function

Execution path

```
1. Call withdraw(uint)  
2. Check if credit[msg.sender] is greater than or equal to the provided amount.  
3. Decrease the credit value of msg.sender by the amount.  
4. Call require(msg.sender.call.value(amount)).
```

Parameters

amount

Exploitation steps

- Call withdraw function with an arbitrary amount
- Observe the exposed stack trace

Conditions

An attacker should have a means to call the 'withdraw' function.

Remediation

Remove unnecessary debugging statements like 'require(msg.sender.call.value(amount))' in production.

Secure example

```
// In secure code
withdraw(uint amount) internal {
    require(credit[msg.sender] >= amount, "Insufficient credit to withdraw");
    credit[msg.sender] -= amount;
}
```

File 2: /work/project/dataset/clean/modifier_reentrancy_fixed.sol

Similarity score: 0.564

Source lines: 1-42

► Finding 2.1: Vulnerability found (Medium)

Vulnerable code

```
supportsToken() external returns(bytes32)
{ return(keccak256(abi.encodePacked("Nu Token"))); }
```

Explanation

This function call exposes a hardcoded abi.encodePacked string, potentially leaking sensitive information in production.

Impact

Information disclosure

Entry point

supportsToken() function

Execution path

```
'''
1. Call supportsToken()
2. Function call exposes hardcoded abi.encodePacked string
'''
```

Exploitation steps

- Call supportsToken function in production

Conditions

Function call is made in production environment

Remediation

Remove hardcoded abi.encodePacked strings or implement proper input handling

Secure example

```
function supportsToken() internal returns(bytes32)
{ return(keccak256(abi.encodePacked(_tokenName))); }
```

File 3: /work/project/dataset/clean/mycontract_fixed.sol

Similarity score: 0.548

Source lines: 1-31

► Finding 3.1: Vulnerability found (Medium)

Vulnerable code

```
msg.sender == owner
```

Explanation

The contract is comparing msg.sender with the owner variable, exposing the caller's address in production.

Impact

Disclosure of sensitive information

Entry point

sendTo function

Execution path

```
'''  
start -> sendTo -> require(msg.sender == owner) -> expose address  
'''
```

Parameters

receiver, amount

Exploitation steps

- Call the sendTo function with any valid receiver and amount

Remediation

Replace msg.sender with owner before comparing.

Secure example

```
require(msg.sender == owner.call().addr)
```

Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)